CS 216: Bitcoin Transaction Lab

Topic: Legacy (P2PKH) and SegWit (P2SH-P2WPKH) Transaction Analysis

Team Name: **Strangerbitz**

Team Member1:

Name: Chandana Lakshmi Subhadra Roll No: 240041009

Team Member2:

Name: Gopisetti Pradhyumna Roll No: 240041018

Team Member3:

Name: Gujjula Siri Sahasra Roll No: 240001032

Team Member4:

Name: Menni Hima Harika Roll No: 240001046

Part 1: Legacy Transactions (P2PKH)

Workflow

The experiment was performed on the **Bitcoin regtest network.** Three nodes were simulated using three addresses: **A, B, and C**.

Step 1: Address Creation

Three legacy addresses were generated:

Node	Address
A	mySmw8yS7HK5Mz3HHSXWeGt8dbCzUgApe3
B	mwdYQJbog3sKqGFYXrxtj5eC5HSAdekgkb
C	mjXKjbsRYQA7H1SLg3D8mfxzEEzxHa1hzY

Address **A** acts as the source node that initially receives mined coins.

Step 2: Mining Blocks

To generate spendable bitcoins, **101 blocks were mined** to address **A**.
In Bitcoin, newly mined coins can only be spent after **100 confirmations**.

After mining, the wallet received a **UTXO (Unspent Transaction Output)**.

Example UTXO:

txid: 0a253a6816f0e33c05fe6c97f92cd343209f0b29a95662c9aea5a343fee6f693
amount: 5 BTC

Step 3: Transaction A → B

The UTXO belonging to **A** was used as input for a new transaction sending funds to **B**.

Transaction ID: 781f6edac495c2afb5744e2ad32c0848e5fae7c227ad78a44a0281eb5474e206

Transaction details:

Output	**Amount**
B	2.5 BTC
Change back to A	2.4999 BTC

After broadcasting the transaction, a new block was mined to confirm it.

SCREENSHOTS:

Step 4: Transaction B → C

In Bitcoin, **the output of a previous transaction becomes a new UTXO that can be used as input for another transaction**.

The wallet selects an available UTXO and uses it as the input for the next transaction sending funds from **B to C**.

Transaction ID:

278b8208ffbf74f46efd000f6ba8a959ca6ece7b12e28b84ab93ae20b5944d29

Transaction details:

Output	Amount
C	72.49998590 BTC
Change	72.49988590 BTC

This demonstrates the **UTXO chaining mechanism** in Bitcoin where the output of one transaction becomes the input of another.

SCREENSHOTS:

```
┌──(sumitmodanwal㉿kali)-[~]
└─$ btcdeb 76a9146f2a26a47764f0ba25cd1fe841f0ffd12228e80688ac 3044022011419b6583ad82fdf7726f18ca9b97c7d44c350d5bbbf6de6866da5bf2b1c8eb0220125067ab67007c93512
09db6e17ec51f15ac3a4e0c505462431c967847d0c48301 02cf139eca7fec23af1900ae291bd5a10801b63058dcb3ec974eb632aab3461db9
btcdeb 5.0.24 -- type `btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
5 op script loaded. type `help` for usage information
script                                  |                                                                  stack
----------------------------------------+------------------------------------------------------------------
OP_DUP                                  | 02cf139eca7fec23af1900ae291bd5a10801b63058dcb3ec974eb632aab3461db9
OP_HASH160                              | 3044022011419b6583ad82fdf7726f18ca9b97c7d44c350d5bbbf6de6866da5...
6f2a26a47764f0ba25cd1fe841f0ffd12228e806 |
OP_EQUALVERIFY                          |
OP_CHECKSIG                             |
#0000 OP_DUP
btcdeb> step
                <> PUSH stack 02cf139eca7fec23af1900ae291bd5a10801b63058dcb3ec974eb632aab3461db9
script                                  |                                                                  stack
----------------------------------------+------------------------------------------------------------------
OP_HASH160                              | 02cf139eca7fec23af1900ae291bd5a10801b63058dcb3ec974eb632aab3461db9
6f2a26a47764f0ba25cd1fe841f0ffd12228e806 | 02cf139eca7fec23af1900ae291bd5a10801b63058dcb3ec974eb632aab3461db9
OP_EQUALVERIFY                          | 3044022011419b6583ad82fdf7726f18ca9b97c7d44c350d5bbbf6de6866da5...
OP_CHECKSIG                             |
#0001 OP_HASH160
btcdeb> step
                <> POP  stack
                <> PUSH stack 6f2a26a47764f0ba25cd1fe841f0ffd12228e806
script                                  |                                                                  stack
----------------------------------------+------------------------------------------------------------------
6f2a26a47764f0ba25cd1fe841f0ffd12228e806 |                                  6f2a26a47764f0ba25cd1fe841f0ffd12228e806
OP_EQUALVERIFY                          | 02cf139eca7fec23af1900ae291bd5a10801b63058dcb3ec974eb632aab3461db9
OP_CHECKSIG                             | 3044022011419b6583ad82fdf7726f18ca9b97c7d44c350d5bbbf6de6866da5...
#0002 6f2a26a47764f0ba25cd1fe841f0ffd12228e806
btcdeb> step
                <> PUSH stack 6f2a26a47764f0ba25cd1fe841f0ffd12228e806
script                                  |                                                                  stack
----------------------------------------+------------------------------------------------------------------
```

```
----------------------------------------+-----------------------------------------------------------------
OP_EQUALVERIFY                          |                          6f2a26a47764f0ba25cd1fe841f0ffd12228e806
OP_CHECKSIG                             |                          6f2a26a47764f0ba25cd1fe841f0ffd12228e806
                                        | 02cf139eca7fec23af1900ae291bd5a10801b63058dcb3ec974eb632aab3461db9
                                        | 3044022011419b6583ad82fdf7726f18ca9b97c7d44c350d5bbbf6de6866da5...

#0003 OP_EQUALVERIFY
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
script                                  |                                                              stack
----------------------------------------+-----------------------------------------------------------------
OP_CHECKSIG                             | 02cf139eca7fec23af1900ae291bd5a10801b63058dcb3ec974eb632aab3461db9
                                        | 3044022011419b6583ad82fdf7726f18ca9b97c7d44c350d5bbbf6de6866da5...

#0004 OP_CHECKSIG
btcdeb> step
EvalChecksig() sigversion=0
Eval Checksig Pre-Tapscript
error: Signature must be zero for failed CHECK(MULTI)SIG operation
btcdeb>
```

Decoded Scripts (P2PKH)

Locking Script (scriptPubKey):
76a914c4a7f39f986ecd73cedb63e888692a933150f37f88ac

Decoded Script:
OP_DUP

OP_HASH160 < c4a7f39f986ecd73cedb63e888692a933150f37f>

OP_EQUALVERIFY

OP_CHECKSIG

Unlocking Script (scriptSig):
<signature> <public_key>

Challenge–Response Script Validation

In Bitcoin transactions, the locking script acts as a challenge and the unlocking script acts as a response. The unlocking script pushes the digital signature and public key onto the stack. The locking script then performs operations such as OP_DUP and OP_HASH160 to verify that the provided public key matches the hash stored in the transaction output. Finally, OP_CHECKSIG verifies the digital signature. If all checks succeed, the script evaluates to TRUE and the transaction is considered valid.

Part 2: SegWit Transactions (P2SH-P2WPKH)

Workflow

In the SegWit experiment, three SegWit addresses A', B', and C' were generated using P2SH-P2WPKH format. SegWit separates signature data from the main transaction structure and stores it in the witness field.

Node	Address
A'	2Msnc5CQ8iyKYYvVnaGQg8gUYskse4m1bVQ
B'	2NDawZ2YEmBopUZ42KZyp8iD94s2KcHDQ2X
C'	2NGJMMxPPx1WdFBZ1Nwmy5Rvv2oXHci72ey

Transaction A' → B'
TXID: 575cdd4d17b10bf31799d0ef8f13a8cf392a96ad1959bbe10bdb9d4c7c5e52bf
The UTXO belonging to address A' was used as input. The transaction transferred funds to B' and returned the remaining balance as change.

Output	Amount
B'	3BTC
Change	1.9999 BTC

Unlike legacy transactions, the signature data is placed in the **witness field**.

SCREENSHOTS:

```
┌──(sumitmodanwal㉿kali)-[~]
└─$ btcdeb a91405ef548862db266670bd2c7f155f72824764d52c87 0014ee31236d6e25edfd886a06f5733ebd233103eaf1
btcdeb 5.0.24 -- type `btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
6 op script loaded. type `help` for usage information
script                                   |                                        stack
-----------------------------------------+-----------------------------------------------
OP_HASH160                               | 0014ee31236d6e25edfd886a06f5733ebd233103eaf1
05ef548862db266670bd2c7f155f72824764d52c |
OP_EQUAL                                 |
<<< P2SH script >>>                      |
0                                        |
ee31236d6e25edfd886a06f5733ebd233103eaf1 |
#0000 OP_HASH160
btcdeb> step
                <> POP  stack
                <> PUSH stack 05ef548862db266670bd2c7f155f72824764d52c
script                                   |                                        stack
-----------------------------------------+-----------------------------------------------
05ef548862db266670bd2c7f155f72824764d52c |     05ef548862db266670bd2c7f155f72824764d52c
OP_EQUAL                                 |
<<< P2SH script >>>                      |
0                                        |
ee31236d6e25edfd886a06f5733ebd233103eaf1 |
#0001 05ef548862db266670bd2c7f155f72824764d52c
btcdeb> step
                <> PUSH stack 05ef548862db266670bd2c7f155f72824764d52c
script                                   |                                        stack
-----------------------------------------+-----------------------------------------------
```

```
----------------------------------------+-------------------------------------------
OP_EQUAL                                |     05ef548862db266670bd2c7f155f72824764d52c
<<< P2SH script >>>                     |     05ef548862db266670bd2c7f155f72824764d52c
0                                       |
ee31236d6e25edfd886a06f5733ebd233103eaf1 |
#0002 OP_EQUAL
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
script                                  |                                       stack
----------------------------------------+-------------------------------------------
<<< P2SH script >>>                     |                                          01
0                                       |
ee31236d6e25edfd886a06f5733ebd233103eaf1 |
<<< P2SH script >>>
btcdeb>
```

Transaction B' → C'

TXID: 855ff27ac8717d45534bf3e2c4244ef6be25ecad1c93b7be532de4f9de9cb706

The output received by B' became the input for the next transaction sending funds to address C'.

Output	Amount
C'	12.5 BTC
Change	12.4999 BTC

SCREENSHOTS:

```
┌──(sumitmodanwal㉿kali)-[~]
└─$ btcdeb a91405ef548862db266670bd2c7f155f72824764d52c87 00149bdd279a93c8800a72ffdb6cbb476d10f529b73d
btcdeb 5.0.24 -- type `btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
6 op script loaded. type `help` for usage information
script                                  |                                       stack
----------------------------------------+-------------------------------------------
OP_HASH160                              | 00149bdd279a93c8800a72ffdb6cbb476d10f529b73d
05ef548862db266670bd2c7f155f72824764d52c |
OP_EQUAL                                |
<<< P2SH script >>>                     |
0                                       |
9bdd279a93c8800a72ffdb6cbb476d10f529b73d |
#0000 OP_HASH160
btcdeb> step
                <> POP  stack
                <> PUSH stack 2b03a0e1c079013de0d4f7f865a81e5aada69b97
script                                  |                                       stack
----------------------------------------+-------------------------------------------
05ef548862db266670bd2c7f155f72824764d52c |     2b03a0e1c079013de0d4f7f865a81e5aada69b97
OP_EQUAL                                |
<<< P2SH script >>>                     |
0                                       |
9bdd279a93c8800a72ffdb6cbb476d10f529b73d |
#0001 05ef548862db266670bd2c7f155f72824764d52c
btcdeb> step
                <> PUSH stack 05ef548862db266670bd2c7f155f72824764d52c
script                                  |                                       stack
----------------------------------------+-------------------------------------------
```

```
----------------------------------------+-------------------------------------------
OP_EQUAL                                |     05ef548862db266670bd2c7f155f72824764d52c
<<< P2SH script >>>                     |     2b03a0e1c079013de0d4f7f865a81e5aada69b97
0                                       |
9bdd279a93c8800a72ffdb6cbb476d10f529b73d |
#0002 OP_EQUAL
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack
script                                  |                                     stack
----------------------------------------+-------------------------------------------
<<< P2SH script >>>                     |                                        0x
0                                       |
9bdd279a93c8800a72ffdb6cbb476d10f529b73d |
<<< P2SH script >>>
btcdeb>
```

SegWit Script Structure

Locking Script (scriptPubKey): OP_HASH160 <script_hash> OP_EQUAL

Redeem Script: 0 <public_key_hash>

Witness Data:
<signature>
<public_key>

In SegWit transactions, the signature and public key are stored in the witness field instead of the scriptSig field.

Part 3: Analysis and Comparison

Transaction Size Comparison

Transaction Type	Size	vsize	Weight
P2PKH	~222 bytes	222	888
P2SH-P2WPKH	~223 bytes	142	565

Although the raw size looks similar, the **virtual size is smaller in SegWit**.

Script Structure Comparison

P2PKH Script Structure:
scriptSig: <signature> <public_key>
scriptPubKey: OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG

P2SH-P2WPKH Script Structure:
scriptSig: redeem script
scriptPubKey: OP_HASH160 <script_hash> OP_EQUAL
witness: <signature> <public_key>

Why SegWit Transactions are Smaller

SegWit separates signature data from the main transaction. Witness data is discounted when calculating block weight. Because of this, SegWit transactions have a smaller effective virtual size (vsize) compared to legacy transactions.

Benefits of SegWit

1. Reduced transaction size
2. Lower transaction fees
3. Increased block capacity
4. Fixes transaction malleability
5. Enables advanced scaling solutions such as the Lightning Network

Conclusion

This experiment demonstrated how Bitcoin transactions operate using both **legacy P2PKH** and **SegWit P2SH-P2WPKH** formats.

The workflow showed how outputs from one transaction become inputs for another using the **UTXO model**.

Script analysis illustrated the **challenge-response validation mechanism** used in Bitcoin Script.

Finally, the comparison confirmed that **SegWit transactions are more efficient**, reducing effective transaction size and improving network scalability.